

JAVA PROGRAMMING LABORATORY

Experiment No: 05

Date: 21/09/2026

Title: Demonstration of NavigableMap Interface Methods using TreeMap

1. Objective / Aim

To understand the NavigableMap interface in Java and demonstrate commonly used NavigableMap methods such as lowerKey(), floorKey(), ceilingKey(), higherKey(), firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry(), and descendingMap() using TreeMap.

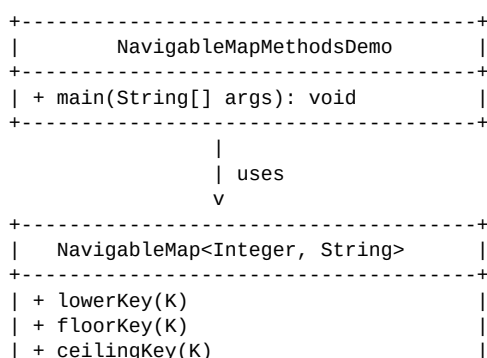
2. System Requirements

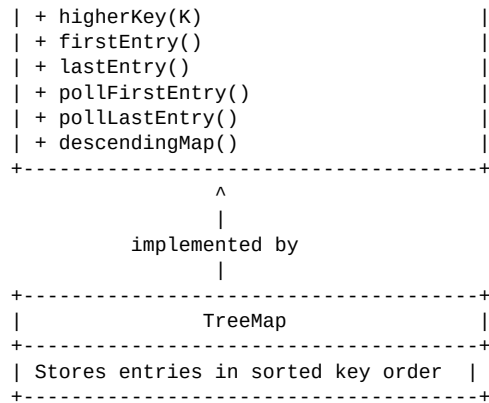
Hardware Requirements	Software Requirements
Computer/Laptop	Operating System: Windows / Linux / macOS
Minimum 4 GB RAM	Java Development Kit (JDK) 17 or above
Minimum 1 GHz processor	IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
At least 100 MB free storage	Java compiler and Command Prompt/Terminal

3. Algorithm / Logic Description

1. Import NavigableMap and TreeMap from java.util.
2. Create a NavigableMap using TreeMap.
3. Insert student roll numbers and names as key-value pairs.
4. Display the complete NavigableMap in ascending key order.
5. Use lowerKey(105) to find the greatest key strictly less than 105.
6. Use floorKey(105) to find the greatest key less than or equal to 105.
7. Use ceilingKey(106) to find the smallest key greater than or equal to 106.
8. Use higherKey(105) to find the smallest key strictly greater than 105.
9. Use firstEntry() and lastEntry() to obtain the first and last key-value entries.
10. Use pollFirstEntry() to retrieve and remove the first entry.
11. Use pollLastEntry() to retrieve and remove the last entry.
12. Use descendingMap() to display the remaining entries in descending key order.

4. UML Class Diagram





5. Source Code

```
import java.util.NavigableMap;
import java.util.TreeMap;

class NavigableMapMethodsDemo {

    public static void main(String[] args) {

        // Creating a NavigableMap using TreeMap
        NavigableMap<Integer, String> students = new TreeMap<>();

        // Adding key-value pairs
        students.put(101, "Aarav");
        students.put(103, "Rohan");
        students.put(105, "Diya");
        students.put(107, "Anjali");
        students.put(109, "Meera");

        System.out.println("NavigableMap: " + students);

        // lowerKey()
        System.out.println("Lower Key of 105: " + students.lowerKey(105));

        // floorKey()
        System.out.println("Floor Key of 105: " + students.floorKey(105));

        // ceilingKey()
        System.out.println("Ceiling Key of 106: " + students.ceilingKey(106));

        // higherKey()
        System.out.println("Higher Key of 105: " + students.higherKey(105));

        // firstEntry()
        System.out.println("First Entry: " + students.firstEntry());

        // lastEntry()
        System.out.println("Last Entry: " + students.lastEntry());

        // pollFirstEntry()
        System.out.println("Poll First Entry: " + students.pollFirstEntry());
        System.out.println("After pollFirstEntry(): " + students);

        // pollLastEntry()
        System.out.println("Poll Last Entry: " + students.pollLastEntry());
        System.out.println("After pollLastEntry(): " + students);

        // descendingMap()
        System.out.println("Descending Map: " + students.descendingMap());

    }
}
```

6. Completion and Execution Commands

Step 1: Save the program as:
NavigableMapMethodsDemo.java

Step 2: Compile:
javac NavigableMapMethodsDemo.java

Step 3: Execute:

```
java NavigableMapMethodsDemo
```

7. Output

```
NavigableMap: {101=Aarav, 103=Rohan, 105=Diya, 107=Anjali, 109=Meera}  
Lower Key of 105: 103  
Floor Key of 105: 105  
Ceiling Key of 106: 107  
Higher Key of 105: 107  
First Entry: 101=Aarav  
Last Entry: 109=Meera  
Poll First Entry: 101=Aarav  
After pollFirstEntry(): {103=Rohan, 105=Diya, 107=Anjali, 109=Meera}  
Poll Last Entry: 109=Meera  
After pollLastEntry(): {103=Rohan, 105=Diya, 107=Anjali}  
Descending Map: {107=Anjali, 105=Diya, 103=Rohan}
```

8. Result / Completion

The program was successfully compiled and executed. The NavigableMap interface methods were demonstrated using TreeMap, including navigation, entry retrieval, entry removal, and descending-order traversal.